AI-Built Mac PDF App Gameplan

> **For Hermes:** Use software-development-lifecycle / subagent-driven development to implement this plan task-by-task.

Goal: Build a native macOS PDF utility app with a similar capability set to PDF Expert: reading, editing, annotation, page organization, conversion, forms, signatures, redaction, OCR, scan enhancement, compression, and page extraction.

Architecture: Native macOS application with a SwiftUI shell, AppKit/PDFKit for document rendering and manipulation, Vision for OCR, Core Image/Core Graphics for image processing, and isolated services for file conversion and signing. Use a local-first design: documents stay on the Mac unless the user explicitly exports or shares them.

Tech Stack: Swift 6, SwiftUI, AppKit, PDFKit, Vision, Core Image, Core Graphics, XCTest, XCUITest, Swift Package Manager, Xcode. Optional later integrations: LibreOffice headless conversion, qpdf, Ghostscript, or sandboxed helper processes.

---

0. Product boundaries and non-negotiable principles

1. Build an original product and interface. Do not copy PDF Expert's branding, assets, exact layout, wording, or proprietary implementation.
2. Define a small MVP before attempting the full feature list.
3. Keep the app local-first and privacy-preserving by default.
4. Never overwrite the original document without an explicit user action.
5. Every destructive operation must support Undo where technically possible, or create a new output document.
6. Treat imported PDFs as untrusted input. Avoid executing embedded JavaScript, launching links automatically, or trusting metadata.
7. AI writes the code, but every milestone must be compiled, tested, and manually smoke-tested by the controlling workflow.
8. Use test-first development for each behavior: write a failing test, verify the failure, implement the smallest fix, then run regression tests.

---

1. Decide the MVP

MVP release target

Ship a dependable Mac PDF reader and organizer first:

1. Open, close, and reopen PDFs
1. Recent documents
1. Page thumbnails and page navigation

1. Single-page, two-page, and continuous viewing
1. Day, Night, and Sepia reading themes
1. Text search within a PDF
1. Add, delete, rotate, reorder, extract, and merge pages
1. Export the result as a new PDF
1. Undo/redo for page operations
1. Save/export using a safe temporary-file strategy

Defer until after MVP

1. Full text editing
1. Image and link editing
1. Broad format conversion
1. Fillable forms
1. Digital signatures
1. Redaction
1. OCR and scan enhancement
1. Compression optimization
1. Cloud sync
1. Collaboration
1. AI document features

This prevents the project from becoming a PDF-shaped swamp monster before the document engine is trustworthy.

---

2. Resolve technical feasibility with spikes

Create disposable prototypes under `spikes/` before committing to the production architecture.

Spike A — PDFKit baseline

Validate:

1. Rendering large PDFs
1. Page thumbnails
1. Search/indexing behavior
1. Page insertion, deletion, rotation, and reordering
1. Undo/redo strategy
1. Performance with 500+ page documents

Deliverable: runnable prototype plus a README verdict: `VALIDATED`, `PARTIAL`, or `INVALIDATED`.

Spike B — PDF text editing

Test whether PDFKit can reliably edit existing text. It generally does not provide true arbitrary text-layout editing, so compare approaches:

1. Overlay editable text annotations
1. Rebuild selected pages
1. Integrate a commercial/native PDF engine
1. Use a lower-level PDF library

Do not promise Word-processor-like editing until a prototype proves it.

Spike C — Format conversion

Test representative files for:

1. DOCX → PDF
1. PPTX → PDF
1. XLSX → PDF
1. PDF → DOCX
1. PDF → image
1. PDF → TXT

Determine which conversions can be native and which require an optional helper such as LibreOffice, qpdf, or Ghostscript. Document licensing, installation, sandboxing, and failure behavior.

Spike D — OCR and enhancement

Validate Vision OCR on:

1. Clean digital scans
1. Skewed scans
1. Low-contrast scans
1. Multi-column pages
1. Handwriting, if supported as a best-effort feature

Measure accuracy, processing time, memory use, and whether OCR text can be embedded into an output PDF.

Spike E — Signatures and redaction

Prototype:

1. Drawing a signature with mouse/trackpad
1. Importing a signature image
1. Persisting a signature securely
1. Applying a signature appearance to a page
1. Permanently removing/redacting content rather than merely covering it

The redaction spike must verify that hidden text cannot be selected, copied, searched, or extracted afterward.

---

3. Create the repository and project foundation

Suggested repository structure:

```
PDFStudio/
├── App/
│   ├── PDFStudioApp.swift
│   ├── AppState.swift
│   └── Commands.swift
├── Features/
│   ├── Reader/
│   ├── Search/
│   ├── Pages/
│   ├── Editing/
│   ├── Annotation/
│   ├── Forms/
│   ├── Signing/
│   ├── Redaction/
│   ├── OCR/
│   ├── Conversion/
│   └── Export/
├── Core/
│   ├── DocumentStore/
│   ├── PDFEngine/
│   ├── Undo/
│   ├── Security/
│   └── Diagnostics/
├── SharedUI/
├── Resources/
├── Tests/
│   ├── PDFEngineTests/
│   ├── FeatureTests/
│   └── PDFStudioUITests/
├── Fixtures/
├── spikes/
└── README.md
```

Foundation tasks:

1. Create an Xcode macOS app project.
2. Enable App Sandbox with only required file permissions.
3. Add document-based app behavior or a controlled document-open flow.
4. Define Swift concurrency rules and actor isolation.
5. Add a logging system that never logs document contents or signatures.
6. Add CI for build, unit tests, UI tests where supported, and static checks.
7. Add a decision log for architecture choices.

Verification command examples:

```
xcodebuild -scheme PDFStudio -destination 'platform=macOS' build
xcodebuild -scheme PDFStudio -destination 'platform=macOS' test
```

---

4. Build the document core first

Implement these components before polishing UI:

DocumentStore

Responsibilities:

1. Open security-scoped URLs
1. Load PDFs safely
1. Track active document state
1. Track dirty/unsaved state
1. Save to a new file or explicitly replace the original
1. Handle cancellation and errors
1. Release security-scoped resources

PDFEngine

Responsibilities:

1. Page count and page access
1. Rendering pages at a requested scale
1. Page insertion/deletion
1. Page rotation
1. Page ordering
1. Page extraction
1. PDF merging
1. Metadata handling
1. Exporting a new PDF

Operation model

Represent page mutations as typed operations, for example:

1. `InsertPages`
1. `DeletePages`

1. `MovePages`
1. `RotatePages`
1. `MergeDocuments`
1. `ExtractPages`

Each operation should be testable without the UI and reversible where possible.

Core tests

Create fixture PDFs and test:

1. Page count before and after every operation
1. Correct page order
1. Correct rotation angle
1. Original file remains unchanged until explicit replacement
1. Exported files reopen successfully
1. Invalid page selections return typed errors
1. Large documents do not block the main actor

---

5. Implement the reader experience

Build the reader around PDFKit's `PDFView`, wrapped cleanly for SwiftUI.

Reader features

1. Open a PDF through Finder, drag-and-drop, or the app's Open command.
2. Display a toolbar with page number, zoom, search, sidebar, and share/export actions.
3. Support continuous scrolling and page-by-page viewing.
4. Add single-page and two-page layouts.
5. Add fit-to-width and fit-to-page zoom modes.
6. Add Day, Night, and Sepia themes.
7. Add a page thumbnail sidebar.
8. Add keyboard shortcuts for navigation and zoom.
9. Preserve view state per document where appropriate.
10. Add clear empty, loading, and error states.

Reader tests

1. Opening a fixture PDF displays the correct page count.

1. Navigation reaches first, last, and selected pages.
1. Theme changes affect the reader without modifying document content.
1. Closing a dirty document prompts appropriately.
1. A corrupt or password-protected PDF produces a useful error.

---

6. Add search

Implement search in two stages:

1. Use PDFKit's document search for the MVP.
2. Add a background index only if performance tests show it is needed.

Requirements:

1. Search current document
1. Navigate between matches
1. Show match count
1. Highlight results
1. Search case-insensitively by default
1. Handle textless scanned PDFs with a clear “run OCR” path
1. Never freeze the reader during a search

Tests should cover no matches, many matches, Unicode text, rotated pages, and scanned pages.

---

7. Add page management

Create a thumbnail-based page organizer.

Features:

1. Multi-select pages
1. Insert pages from another PDF
1. Delete pages
1. Rotate selected pages clockwise/counterclockwise
1. Drag to reorder pages
1. Extract selected pages to a new PDF
1. Merge another PDF into the current document
1. Duplicate selected pages if supported by the engine

1. Undo/redo every mutation
1. Export as a new PDF

Safety requirements:

1. Confirm deletion when appropriate.
1. Prevent accidental loss of the only page.
1. Use atomic writes: write to a temporary file, validate it, then move it into place.
1. Keep original and output URLs distinct.

Integration tests should reopen every generated PDF and verify page count, order, and rotations.

---

8. Add annotations

Implement annotations independently from true text editing.

Initial annotation set:

1. Highlight
1. Underline
1. Strikethrough
1. Freehand drawing
1. Text note
1. Pop-up note
1. Rectangle, oval, line, and arrow
1. Built-in stamps
1. Custom stamps, if time permits

Requirements:

1. Select, move, resize, and delete annotations.
1. Persist annotations through save/export.
1. Support Undo/redo.
1. Use accessible labels and keyboard alternatives where feasible.
1. Test annotations on rotated pages and at different zoom levels.

Important distinction: annotation overlays are not the same thing as editing the original PDF text.

---

9. Add forms

Start with AcroForm support.

Features:

1. Detect fillable fields
1. Text fields
1. Checkboxes
1. Radio buttons
1. Dropdowns
1. Date fields where supported
1. Keyboard tab order
1. Reset form
1. Export filled PDF
1. Flatten a copy for sharing

Research XFA forms separately; do not claim support unless fixtures prove it.

Tests:

1. Field discovery
1. Correct value persistence
1. Required-field validation
1. Export/reopen behavior
1. Flattening removes interactive behavior while preserving appearance

---

10. Add signatures

Implement a simple, honest signature workflow:

1. Let the user draw a signature with trackpad/mouse.
2. Allow importing a transparent signature image.
3. Let the user place, resize, and rotate the signature on a page.
4. Save the visual signature into a new PDF.
5. Offer optional local storage protected by the macOS Keychain.
6. Make it clear whether the result is a visual signature or a cryptographic digital signature.

Do not describe a visual signature as a legally verified digital signature.

Future phase: investigate certificate-based PDF signing with a vetted library and security review.

---

11. Add permanent redaction

This feature requires extra caution.

Workflow:

1. User marks regions for redaction.
2. UI displays a clear redaction preview.
3. On export, remove or rasterize the underlying content in those regions.
4. Add an optional redaction reason label.
5. Reopen the exported PDF and automatically test that the removed text cannot be searched, selected, copied, or extracted.
6. Warn users that the original document is not modified unless they explicitly choose replacement.

Never implement redaction as a black rectangle placed over existing text.

---

12. Add OCR

Pipeline:

1. Render each target page to an image at a controlled DPI.
2. Run Vision text recognition off the main actor.
3. Collect recognized text, bounding boxes, confidence, and page coordinates.
4. Allow the user to review or rerun OCR.
5. Embed an invisible text layer into a copy of the PDF.
6. Make the result searchable, highlightable, and copyable.
7. Show progress, cancellation, and failure details.

Performance requirements:

1. Process pages incrementally.
1. Avoid holding all high-resolution page images in memory.
1. Support cancellation.
1. Persist only derived OCR data, not unnecessary duplicate page images.

Test with clean scans, skewed scans, low contrast, multiple columns, and mixed-language documents.

---

13. Add scan enhancement and cropping

Enhancement pipeline options:

1. Deskew
1. Perspective correction
1. Shadow removal
1. Contrast adjustment
1. Sharpening
1. Background whitening
1. Noise reduction
1. Crop margins
1. Split double-page scans

Use Core Image/Core Graphics first. Only add a machine-learning model if deterministic processing is insufficient and the model's license, size, privacy, and performance are acceptable.

Always show before/after preview and export a copy rather than silently replacing the original.

---

14. Add conversion and compression

Treat conversion as a separate subsystem because it has the highest dependency and compatibility risk.

First-party/native conversions

Implement and test:

1. PDF → image
1. PDF → text
1. Images → PDF
1. PDF page extraction

External conversions

Evaluate optional helpers for:

1. DOCX → PDF
1. PPTX → PDF
1. XLSX → PDF

1. PDF → DOCX
1. PDF → Excel
1. PDF → PowerPoint

For every helper, document:

1. License compatibility
1. Installation method
1. Sandboxing strategy
1. Whether the helper is bundled or user-installed
1. Privacy implications
1. Error messages
1. Behavior when the helper is unavailable

Compression

Create profiles such as:

1. Lossless / maximum quality
1. Balanced
1. Smallest file

Measure output quality, file size, embedded fonts, images, annotations, forms, and OCR layers. Never claim that compression preserves everything until fixture tests prove it.

---

15. Build the product UI and workflow polish

After core behavior works, standardize the interface.

Required states:

1. Empty app state
1. Loading state
1. Progress state
1. Cancellation state
1. Permission denied
1. Password-protected document
1. Unsupported file type
1. Corrupt document

1. Missing conversion dependency
1. Export success
1. Export failure
1. Unsaved changes

Accessibility:

1. VoiceOver labels
1. Keyboard navigation
1. Sufficient contrast
1. Do not rely on color alone for annotation or redaction state
1. Respect Reduce Motion
1. Use native macOS conventions where possible

Add onboarding only after the core workflow is understandable without it.

---

16. AI-only coding workflow

The app can be written entirely by AI, but the process must be constrained.

For every task:

1. Give the coding agent one narrow objective.
2. Include exact files, acceptance criteria, and test commands.
3. Require a failing test before production code.
4. Require the agent to run the test and report actual output.
5. Require a small diff; avoid unrelated refactors.
6. Have a separate review pass check specification compliance.
7. Have a separate review pass check security, concurrency, performance, and maintainability.
8. Run the complete build and test suite after each milestone.
9. Keep a changelog and architecture decision record.
10. Do not accept “implemented” without a build artifact or test result.

Suggested AI task order:

1. Project shell and CI
2. DocumentStore
3. PDFEngine page operations
4. Reader UI

5. Search
6. Page organizer
7. Export and atomic save
8. Annotations
9. Forms
10. Signatures
11. OCR
12. Enhancement
13. Native conversions
14. External conversion helpers
15. Compression
16. Accessibility and polish
17. Packaging, signing, and release QA

---

17. Testing strategy

Unit tests

1. Page operations
1. File validation
1. Export paths
1. Annotation geometry
1. Search result handling
1. OCR coordinate conversion
1. Form field mapping
1. Redaction region calculations
1. Compression profiles

Integration tests

1. Open → edit pages → export → reopen
1. Merge → reorder → rotate → export
1. OCR → search → highlight → export
1. Fill form → flatten → reopen
1. Mark redaction → export → verify content is gone
1. Convert → validate generated file

UI tests

1. Open document
1. Navigate pages
1. Search
1. Select and reorder thumbnails
1. Apply annotation
1. Fill form
1. Place signature
1. Export
1. Handle errors and cancellation

Fixture library

Include PDFs with:

1. Text
1. Images
1. Links
1. Rotated pages
1. Annotations
1. Forms
1. Password protection
1. Scanned pages
1. OCR text layers
1. Large page counts
1. Unicode and right-to-left text
1. Malformed or unusual metadata

Performance tests

Track:

1. Time to first page render
1. Memory use with large PDFs
1. Thumbnail generation time
1. Search latency
1. OCR throughput
1. Export time

1. Compression time

---

18. Security and privacy review

Before release, verify:

1. PDFs are treated as untrusted input.
1. Embedded scripts are not executed.
1. External links require an explicit user action.
1. Paths are validated and cannot escape intended directories.
1. Temporary files are created securely and cleaned up.
1. Signatures are not logged.
1. Sensitive document text is not sent to a server by default.
1. OCR and enhancement run locally unless the user explicitly opts into a remote service.
1. Redacted data is actually removed.
1. Conversion helpers cannot silently upload files.
1. Keychain access is limited to signature data or other necessary secrets.
1. Crash reports do not include document contents.

Have an independent security review before shipping redaction, signatures, or external helper processes.

---

19. Release engineering

1. Create a stable release configuration.
2. Enable hardened runtime and appropriate entitlements.
3. Code-sign the application.
4. Notarize it with Apple.
5. Test a clean installation on a supported macOS version.
6. Test upgrades from the previous version.
7. Verify file associations and Finder “Open With.”
8. Verify drag-and-drop permissions.
9. Produce a signed DMG or installer.
10. Add crash reporting only after privacy review.
11. Publish a support page and known-limitations list.
12. Maintain a rollback build.

---

20. Definition of done

The app is ready for an MVP release only when:

1. It builds from a clean checkout.
1. Unit and integration tests pass.
1. UI smoke tests pass on a clean Mac user account.
1. The app can safely open, manipulate, and export fixture PDFs.
1. Original files are protected from accidental overwrite.
1. Errors are understandable and recoverable.
1. No known critical security issue remains.
1. Performance is acceptable on large PDFs.
1. The app is code-signed and notarized.
1. Documentation clearly distinguishes supported, partial, and unsupported features.

For the full product, do not mark a feature complete merely because a button exists. Mark it complete only after it works on representative fixtures, survives export/reopen, handles failure, and has automated coverage.

Recommended first milestone

Build only this vertical slice first:

```
Open PDF → view pages → search text → select pages → reorder/delete/rotate
→ export a new PDF → reopen exported PDF → verify the result
```

If that slice is stable, the rest of the product has a real foundation. If it is unstable, adding OCR, conversion, and signatures will merely give the bugs nicer hats.